

Прогресс по проекту

Что уже сделано

Написали бенчмарк `bench_hf.py` для speculative decoding через HuggingFace Transformers — без SGLang, без серверов, всё крутится прямо в ноутбуке через `model.generate(assistant_model=draft_model)`. Удобно, потому что не надо поднимать отдельный процесс.

Прогнали первый прогон на Google Colab (Tesla T4, 16GB VRAM) с target-моделью **Qwen2.5-1.5B-Instruct** (bf16). Тестировали 4 конфига draft-модели:

- `baseline` — без спекуляции, просто авторегрессия
- `self-4bit` — та же 1.5B, но в 4 как драфт
- `0.5B-fp16` — Qwen2.5-0.5B-Instruct в fp16
- `0.5B-4bit` — та же 0.5B, но в 4bit

`num_assistant_tokens` перебирали: `[0, 3, 5, 8]`, 8 промптов, 256 токенов на генерацию. Метрики: `speed_tok_s`, `acc_length`, `step_time_ms`.

Что получилось: `baseline` даёт ~22.7 tok/s, а speculative варианты на T4 с текущими конфигами оказываются медленнее (~7-10 tok/s). Скорее всего дело в том, что на T4 при одновременной загрузке двух моделей не хватает памяти/bandwidth и GPU не успевает. Построили heatmap speedup по `draft_label × num_assistant_tokens`.

Итого: инфраструктура готова, первый пилот прогнан, результаты есть, но пока неутешительные — нужно правильно подобрать параметры и возможно попробовать sglang (пока не получилось на Yandex DataSphere и Colab запустить).

План дальнейших экспериментов

Общая идея: взять Qwen2.5-0.5B/1.5B/3B/7B, подобрать оптимальные параметры спекулятивного декодинга, потом прогнать полную сетку квантизаций и сравнить метрики.

Неделя 1 (1–7 апреля) — Подбор гиперпараметров и датасет

Разобраться с SpecBench — подходит ли он вообще, или менять на что-то другое. Взять 100–300 примеров для прогонов. Зафиксировать связку `Qwen2.5-0.5B → 0.5B-INT8` как тестовую пару и перебрать `num_assistant_tokens` (примерно 1–10) и `top_k` если есть

смысл. Найти оптимальные параметры по метрике speedup — именно их будем использовать везде дальше.

Неделя 2 (8–14 апреля) — Полная сетка квантизаций

Берём найденные оптимальные параметры и гоняем все комбинации: - Модели: 0.5B, 1.5B, 3B, (7B если влезет в GPU) - Квантизации драфта: fp16, INT8, INT4, (INT2 если есть) - Для каждой пары target+draft записываем: `acc_length`, скорость генерации черновика, end-to-end latency, VRAM usage

Если 7B не влезает — можно попробовать target в 4bit. Если что-то долго гонять — сократить количество примеров.

Неделя 3 (15–21 апреля) — Анализ и выводы

Сравниваем всё что нагоняли. Строим графики: speedup vs quantization, acc_length vs модель, VRAM tradeoff. Смотрим, где квантизация драфта даёт нормальный speedup без сильного падения acc_length. Пишем выводы: какая связка target+draft оптимальна, при каком уровне квантизации ещё есть смысл использовать спекулятивный декодинг.

Команда

Шинделов Олег Шиплюк Михаил Чубаков Вячеслав